Avacado

NOTE

This is in development and is a very early prototype and is subject to change.

Avocado: A Privacy-Preserving Token System

Note: This document outlines a developmental prototype and is subject to change.

Contract Addresses

Contract	Address	Network
EncryptedERC Contract	0x5894792d827D56057718Ca15B266D1A7C4eb3682	Avalanche Fuji Testnet

Our Goal

We operate on the principle that privacy does not equate to secrecy. Our objective is to empower users with control over their financial data while facilitating legitimate oversight.

The Encrypted ERC (EERC) System

The EERC system is designed to preserve privacy in token transactions by encrypting balances on-chain using elliptic curve encryption. Users can demonstrate sufficient balance through zero-knowledge proofs without disclosing the exact amount. For compliance purposes, authorized auditors can decrypt balances using their private key.

While the existing EERC system supports private transfers between registered users, deposit and withdrawal amounts are currently visible on-chain. Our comprehensive implementation aims to achieve end-to-end transaction privacy, ensuring that no amounts are visible at any stage, including during transfers and withdrawals.

Differentiating Features

Our system enables private withdrawals to any external address outside the EERC ecosystem, effectively decoupling encrypted balances from their final destinations. Unlike traditional privacy systems that are confined to their own ecosystems, our solution facilitates private off-ramping.

A unified entry point obfuscates user intent on-chain. All operations are channeled through a single function that intelligently routes to the appropriate handler based on encrypted intent data, preventing observers from discerning the specific operation performed by a user. Withdrawals are processed via a two-phase system: intent submission (Phase 1) followed by execution after a delay (Phase 2). This 24-hour delay mitigates front-running and allows for batch execution, where multiple users' withdrawals are processed collectively, thereby enhancing privacy through anonymity sets.

Batching

We aggregate and execute multiple withdrawal intents in a single batch transaction. Batching enhances privacy by creating anonymity sets. For instance, with 50 intents in a single batch, the linkability between any individual withdrawal and its origin is significantly minimized.

The two-phase delay (24 hours) naturally facilitates the accumulation of intents from multiple users. Batch execution not only reduces gas costs per withdrawal but also amplifies privacy for all participants. The current implementation utilizes browser storage for demonstration purposes, validating the efficacy of the batch execution mechanism.

Enhancing Privacy with Nullifiers

In collaboration with Person, we are implementing an advanced nullifier system. This system proves commitment ownership without exposing critical details, all while maintaining auditor oversight.

Phase 1: Commitment

A user generates a random secret and creates a commitment, which is a hash of the amount, destination, tokenId, nonce, and secret. This commitment, along with encrypted audit data, is then submitted. The encrypted audit data allows auditors to decrypt the user's secret and the associated amount.

Phase 2: Nullifier Reveal

After 24 hours, the user reveals the nullifier, which is a hash of the commitment and secret, along with the amount and destination, supported by a zero-knowledge proof. The public cannot link the nullifier back to the original commitment without the secret.

Privacy Guarantees

- **Privacy:** The public observes unlinked commitments and nullifiers (e.g., 50 within a batch). Auditors, however, can decrypt both phases to link all relevant information (user → commitment → nullifier → amount).
- **Security:** Zero-knowledge proofs prevent fraudulent claims. A nullifier registry prevents double-spending, and the secret serves as a salt, privacy key, and proof of ownership.
- **Result:** This system delivers privacy for the public and compliance capabilities for auditors, offering an optimal balance.

The Auditor System

Auditors possess the capability to decrypt all encrypted balance data and audit trails using their private key. This ensures full compliance with regulatory requirements while preserving privacy from public observation. Future enhancements will include a 7-of-9 threshold encryption scheme, requiring the cooperation of seven out of nine auditors for decryption.

This system assists users in avoiding DEX blocking, where decentralized exchanges may freeze funds from privacy protocols based on transaction history. Our auditor system can verify the legitimacy of funds and ensure regulatory compliance.

Current Progress

- The two-phase withdrawal system is operational and has been successfully tested on the testnet.
- The withdrawals component of the centralized entry point is complete.
- Frontend designs, including a waitlist page, are live at https://avacado.app.
- SDK implementation is in progress, with Phase 2 functions currently undergoing testing.
- All functionalities inherited from the native EERC system are integrated.

Future Developments

- Implementation of a nullifier-based privacy system with commitment-nullifier separation.
- Addition of 7-of-9 threshold encryption for the auditor system.
- Development of Zero-Knowledge circuits for proving commitment membership without revealing the specific commitment.
- Integration of a Merkle tree for commitment storage instead of a mapping to further enhance security.

Two-Phase Private Withdrawal System

Complete Two-Phase Withdrawal Example

Phase 1: Intent Submission

- **Transaction:**
- **From:**
- **To:** (EncryptedERC Contract)
- **Function:** submitWithdrawIntent(tokenId: 1, proof, balancePCT, metadata)
- **Explorer:**

Phase 2: Execution : Anyone can call it

- **Transaction:**
- **Function:** executeWithdrawIntent(...)
- **Time Elapsed:** 7 hours after submission
- **Explorer:**

- **Revealed Values:**
  - **Amount:** 100 TEST tokens
  - **Destination:**

On-Chain (Public) Data

- Your address
- Intent hash (cryptographic commitment)
- Encrypted balance
- ZK proof (764 bytes)

Private Data

- Withdrawal amount
- Destination address
- Nonce

NOTE : we here use the same wallet as it was not possible for us to arrange for 50 intents from 50 diff wallets and we wanted to show the proof of concept

The Two-Phase System

Phase 1: Submit Intent (Private)

`submitWithdrawIntent(tokenId, proof, balancePCT, metadata)`

- Creates: `intentHash = poseidon(amount, destination, tokenId, nonce)`
- Stores: Only the HASH (not the values)
- **Privacy:** Amount and destination are concealed within the hash.

Phase 2: Execute Intent (Public)

What happens next (after 24 hours):

Option A: Individual Execution

`executeWithdrawIntent(intentHash, tokenId, destination, amount, ...)`

- **Privacy:** Low
- **Visibility:** Everyone observes the direct link: YOU → AMOUNT → DESTINATION

Option B: Batch Execution (Recommended)

`executeBatchWithdrawIntents([50 intents])`

- **Privacy:** High
- **Visibility:** Everyone observes 50 withdrawals but cannot link specific users to specific destinations.

How Privacy is Achieved

Phase 1: Submission (Today)

- User1 → intentHash: 0x0f53d067... ❓
- User2 → intentHash: 0x3a7f2e19... ❓
- User3 → intentHash: 0x8c4d1f92... ❓

Observer knows: Three users submitted intents.
Observer doesn't know: The amounts or destinations.

Phase 2: Batch Execution (After 24 hours)

Batch reveals:

- 1000 tokens → 0xAlice
- 500 tokens → 0xBob
- 2000 tokens → 0xCarol

Observer knows: The amounts and destinations.
Observer doesn't know: Which user submitted which withdrawal.

Timeline

- **T+0:** Intent Submitted
  - **TX:** 0x103fa11f...
  - **Intent Hash:** 0x1982ee57...
- **T+7h:** Intent Executed
  - **TX:** 0x6709a969...
  - **Revealed:** 100 tokens → 0xE6670193...
- **Result:** Tokens successfully withdrawn.

Note: The user opted for individual execution after 7 hours rather than awaiting batch execution.

Security Guarantees

1. Immutable Commitment

The contract verifies: `poseidon(amount, destination, salt , tokenId, nonce) == intentHash`.
This ensures that no one can execute the withdrawal with altered parameters; your withdrawal is cryptographically locked to your specified values.

2. Balance Protection

Your balance is locked after intent submission. You cannot submit another intent until the current one is executed, preventing double-spending.

3. Time-Based Access

- **0-24h:** Only you can execute the withdrawal.
- **24h+:** A relayer can batch the withdrawal (recommended for privacy).
- **7d+:** The intent expires and requires resubmission.

What Makes This Different?

Traditional Withdrawal

```
withdraw(amount, destination);
```

- **Privacy:** All transaction details are publicly visible.
- **On-chain:** User → Amount → Destination (ALL PUBLIC)

Our Two-Phase System

Phase 1: Hide the details
`submitWithdrawIntent(proof)` // Only hash visible

Phase 2: Reveal in batch
`executeBatch([many intents])` // Cannot link user to withdrawal

- **Privacy:** Details are hidden during submission.
- **Privacy:** Transactions are mixed with others during execution.
- **On-chain:** Hash is visible in Phase 1; batch anonymity is provided in Phase 2.

How This Example Was Executed

Your Options for Future Withdrawals:

A. Execute yourself (after 1 hour):

- Offers fast withdrawal (as demonstrated in this example).
- Provides lower privacy (reveals the user → amount → destination link).

B. Submit to relayer (after 24 hours, RECOMMENDED):

- Relayer batches your intent with others.
- Provides maximum privacy through anonymity sets.

What Actually Happened:

After 7 hours, the user executed the withdrawal:

```
executeWithdrawIntent(
  intentHash:
"0x1982ee5796a1c70842f7a933130d2d1760046baddce62c9b7176f6810acf439a"
,
  tokenId:     1n, // TEST token
  destination: "0xE6670193988c10D8C1460F118B6223244a4f558A",
  amount:      100n,
  nonce:       1n,
  proof:       { /* ZK proof from Phase 1 */ },
  balancePCT:  [ /* 7 encrypted values */ ],
  metadata:    "0x"
)
```

Result:

- 100 TEST tokens transferred to the specified destination.
- Balance unlocked for subsequent withdrawals.
- Intent marked as executed on-chain.

Key Takeaways

1. **Phase 1 (Submitted):** The amount and destination remain private.
2. **Phase 2 (Pending):** Values will be revealed within a batch.
3. **Privacy:** Achieved by batching multiple users' transactions together.
4. **Security:** Zero-Knowledge proofs cryptographically lock your withdrawal parameters.
5. **Timeline:** Wait 24 hours for relayer batching, or execute yourself after 1 hour.

The greater the number of intents in a batch, the stronger your privacy.